NETSHIELD AI

Understand the system. Explain the evidence.

Internship learning and demo guide | Prepared October 12, 2026

A practical, plain-English companion to the runnable project. Covers the frontend, backend, model training, risk scoring, streaming, storage, performance, Postman and an honest demo script.

Scope: The bundled classifiers are trained on your supplied sample. Full-dataset validation, live packet capture and cloud deployment remain separate steps. This guide distinguishes working features from remaining goals.

1. Your 30-second introduction

"NetShield AI helps a security analyst review network flow records. The backend reads a CSV in small chunks, classifies each valid flow using Random Forest, XGBoost and Extra Trees, checks unusual behaviour using Isolation Forest, and assigns an explained risk priority. Results stream to a dashboard. Analysts can investigate alerts, update incidents and export reports. The included model package is a sample-trained demonstration; full-dataset validation is a separate step."

Memorize the meaning, not every word. If a feature is not implemented, explain the boundary rather than guessing.

2. What problem are we solving?

An analyst may have thousands or millions of network records. Reading each manually is slow. We prioritize flows that look like known attacks or behave unusually, show the evidence and support investigation. The application assists an analyst; it does not prove compromise or block traffic.

A network packet is a small unit of transmitted data. A flow summarizes a related conversation across multiple packets, generally using source/destination address, port, protocol and timing. Your CSV is a set of flow statistics, not raw packet payloads. One CSV row is one record. A flow can contain multiple packets, so 302 rows is not necessarily 302 packets.

The model can classify patterns already represented in its training data. A separate anomaly model can flag unusual patterns. Neither guarantees finding unknown attacks. This implementation classifies current observed traffic; it does not forecast when a future intrusion will happen.

3. Frontend, backend, model and storage

Part	Plain-English job	Technology here
Frontend	The screen and buttons	HTML, CSS and JavaScript
Backend	Validates requests, reads files, scores chunks and sends results	Python, FastAPI
API	The agreed request/response interface	HTTP routes beginning `/api`
ML classifier	Recognizes feature patterns associated with labels	RF, XGBoost, Extra Trees
Anomaly model	Checks whether a flow looks unusual compared with benign training examples	Isolation Forest
Database	Saves accounts, analysis results and analyst actions	SQLite with WAL
Model artifacts	Saved learned parameters and feature order	Trusted joblib bundle
CSV directory	Stores source files and uploads	Configurable `NETSHIELD_DATA`
Transport	Carries live results to the browser	SSE via fetch or polling
Container	Packages the program and dependencies	Docker, optional

Your original frontend used plain HTML; we retained that approach. The mentor's reference lists React/Next.js and PostgreSQL/MongoDB, but this package uses HTML/JS and SQLite. Do not tell an examiner that the listed technologies are all in your running app. Discuss these as deliberate small-project choices and possible migration points.

4. What happens from login to a finished scan?

1. You create an account. Backend stores a salted PBKDF2 password hash, not the password.

2. Login verifies the password and issues a signed JWT valid for two hours. JWT is signed, not encrypted. Never put secret data inside its payload.

3. Browser keeps the access token in session storage and attaches `Authorization: Bearer <token>` to requests. Session storage is still readable by JavaScript; avoid injecting untrusted HTML.

4. Backend checks token signature/expiry, then reads the user's current role from SQLite.

5. Backend source selection returns eligible CSV filenames. Upload copies bytes to disk in 1 MB pieces, with a configurable size limit. An uploaded file receives a generated safe name.

6. Clicking Scan creates a new scan ID and opens the chosen CSV from row one. It does not resume a saved position.

7. Pandas reads a fixed-size chunk, default 200 rows. Required feature columns are checked. Whitespace/case/punctuation aliases are normalized.

8. Numeric feature values are validated. Missing, infinite or nonnumeric values cause that row to be excluded from inference and displayed as invalid. We do not silently replace them with zero.

9. A worker thread scores valid rows with all three classifiers and Isolation Forest. This keeps heavy work off the server's asynchronous event loop.

10. IPs are extracted from source/destination columns and checked as IPv4/IPv6. IPs are evidence, not ML features.

11. Majority vote chooses the consensus label. If each classifier chooses a different class, Random Forest breaks the tie. All predictions remain visible.

12. Risk is computed for each model and for consensus. Uncertainty is separately flagged.

13. Rows are saved to SQLite. Review items are created for High/Critical risk, disagreements, confidence below 0.60 in any model, or anomaly flags.

14. A live message reaches the screen. The browser appends recent rows and removes old display rows beyond 200.

15. This repeats until end of file, a fatal read error or Stop/disconnection.

16. Backend stores a final summary and sends done. Reports and row exports remain available. A new Scan begins from row one.

5. The 15 original numeric features, in simple words

Read `models/metrics.json` for the exact active feature list; new full-data training can change it.

Original sample feature	Meaning
RST Flag Count	Count of TCP reset flags in the summarized flow
Bwd Packet Length Std	Variation in sizes of backward-direction packets
Packet Length Std	Variation in overall packet sizes
Flow Duration	Length of the flow; verify units with your dataset's exporter, commonly microseconds in CICFlowMeter
Flow IAT Max	Largest time gap between flow packets
Bwd Packet Length Mean	Average backward-direction packet size
Packet Length Variance	Another measure of packet-size spread; variance is standard deviation squared
Fwd Packets/s	Forward packets per second
Packet Length Max	Largest packet length
Fwd Segment Size Avg	Average forward segment size as defined by the flow exporter
Subflow Fwd Bytes	Forward bytes attributed to a subflow

Original sample feature	Meaning
Bwd Segment Size Avg	Average backward segment size
Average Packet Size	Exporter's average packet-size statistic
Flow Packets/s	Total flow packet rate
Packet Length Mean	Mean packet length

Forward is the direction used when the exporter establishes the flow; backward is the reverse direction. Do not assume forward always means attacker-to-victim. Definitions and units must come from the exact dataset/exporter. Some features are correlated, which is another reason random row splits can look overly strong.

Label is the answer provided for training/evaluation. It is not an input feature. Source IP, destination IP and timestamp are metadata. A model must not be allowed to "predict" an attack by reading its provided Label.

6. How the classifiers work

Supervised learning: give the algorithm examples with input features and correct labels during training. It learns patterns. During a scan, labels are not supplied to the model.

Decision tree: asks a sequence of numerical questions, such as whether a packet-rate statistic is above a learned threshold. Its leaves predict classes.

Random Forest: trains many trees using randomness in examples and candidate features. Averaging predictions often makes it more robust than one tree. The configured tree count, depth and leaf sizes are hyperparameters.

XGBoost: builds trees sequentially. New trees improve the objective by addressing errors left by previous trees. Learning rate controls how strongly each new tree contributes. Depth and regularization constrain complexity. The integer label encoder translates attack names to numbers for XGBoost; inference maps class numbers back to names.

Extra Trees: another tree ensemble. It randomizes split thresholds more strongly than a conventional random forest. It offers a third opinion and a different speed/quality tradeoff. It is not guaranteed to be better.

All three are compared on the same training/holdout partition and feature schema. It would be unfair to train one on all data and another on a tiny sample, then compare their scores as if conditions were identical.

Ensemble consensus: majority voting is a transparent decision rule, not a new trained probability model. Agreement rate means models often choose the same label; they can all agree and be wrong. Consensus confidence averages only confidence estimates from models that chose the winning label. It is not a calibrated ensemble probability.

Isolation Forest: an unsupervised anomaly model, here fitted only to benign training examples. Random partitions tend to isolate unusual points quickly. We scale features inside a pipeline and use contamination 0.05 as a configured threshold assumption. This does not mean 5% of real-world

traffic is malicious. The output is an anomaly flag, not a named attack or proven compromise.

7. Hyperparameters: what your mentor probably meant

"Hypo parameter" almost certainly meant **hyperparameter**, although we cannot know the exact forgotten context. Your original notebook already used RandomizedSearchCV for RF and XGBoost.

A learned parameter is something the model learns, such as a tree split threshold. A hyperparameter is something we choose before fitting, such as how many trees to build.

Hyperparameter	Plain-English effect
n_estimators	Number of trees; more can improve stability but costs time and memory
max_depth	Maximum tree depth; deeper trees learn finer patterns and can overfit
min_samples_leaf	Minimum examples in a leaf; larger values can reduce overfitting
learning_rate	XGBoost step size; smaller steps usually need more trees
subsample	Fraction of examples used for each boosting stage
colsample_bytree	Fraction of features considered by each boosting tree
class_weight	RF/Extra Trees class balancing to reduce majority-class dominance

`--tune` runs four random parameter combinations per classifier using up to three stratified training folds. It selects using macro F1 and then measures the selected model on untouched holdout rows. Holdout data must not guide tuning or feature selection. These are modest searches intended to remain understandable, not an exhaustive optimization guarantee.

Complexity should serve a purpose: three complementary classifiers, anomaly review, tested streaming and incident workflow. More algorithms do not automatically make a stronger project.

8. Risk scoring: the part you must understand deeply

There are **three different numbers** an examiner might ask about:

• Model confidence: e.g. 0.80 displayed as 80%. The classifier's estimate for the selected class. It may be poorly calibrated.

• Configured severity: e.g. DDoS = 90. A human-written importance policy, not a value learned from CSV training.

• Risk priority: e.g. 72/100. A formula combining the previous two; not an attack probability.

Our compatible policy is `round(severity x confidence)`. Confidence is a fraction from 0 to 1. Thus `90 x 0.80 = 72`. Never multiply by 80 instead of 0.80.

Rounded score	Display category	Meaning here
0 through 25	Low	Lowest band in this prioritization policy
26 through 50	Medium	Middle priority
51 through 75	High	Higher investigation priority
76 through 100	Critical	Highest band

These cutoffs are not universal cybersecurity standards. They are policy boundaries retained from your code. They can be changed after domain review. Python's rounding convention applies before band assignment.

Complete severity policy

Predicted attack class	Severity
BENIGN	0
Portscan	30
Infiltration - Portscan	35
FTP-Patator	45
SSH-Patator	45
Web Attack - Brute Force	50
Web Attack - XSS	55
DoS Slowloris	60
DoS Slowhttptest	60
DoS Hulk	70
DoS GoldenEye	70
Web Attack - SQL Injection	75
Botnet	80
Infiltration	85
DDoS	90
Heartbleed	95
Unknown label	50, explicitly flagged as unknown policy

The relative ordering reflects a simplified teaching policy about potential harm. It is not derived from measured business loss, CVSS, exploit confirmation or a published quantitative risk standard. A production policy should account for asset criticality, actual exposure, exploit success, response costs and calibrated likelihood.

Examples to explain aloud

• BENIGN at 99% confidence: `0 x .99 = 0`, Low. This does not prove safe traffic; anomaly/disagreement can still trigger review.

• Portscan at 100% confidence: `30 x 1 = 30`, Medium. Portscan cannot become Critical under this table alone.

• DDoS at 80%: `90 x .80 = 72`, High.

• DDoS at 90%: `90 x .90 = 81`, Critical.

• Heartbleed at 60%: `95 x .60 = 57`, High, but lower confidence can warrant review.

• Web SQL Injection at 100%: `75 x 1 = 75`, High. If someone expects Critical, explain the configured policy rather than inventing a model reason.

Weakness of the simple formula

Low confidence lowers the score, even for dangerous attack labels. A 30%-confidence Heartbleed result becomes about 28 points; uncertainty should not be interpreted as safety. We therefore add review flags for low confidence, disagreement and anomalies. They do not secretly alter this compatible score. An analyst sees both severity and uncertainty.

Consensus risk uses the winning class's configured severity and the mean confidence among models voting for that class. Individual model risks are also visible. These estimates are not independent; similar tree models share weaknesses. Do not claim majority vote automatically improves detection.

Answer to "Why is it 72%?": "The display should say 72 out of 100, not 72%. DDoS severity is configured as 90 and the classifier confidence is 0.80. Their product is 72, which our policy puts in High. It is a priority score, not the probability of compromise."

9. What "streamable" means

There are two separate streaming problems. First, reading a large CSV without loading all of it. Second, sending scored results without waiting for the whole scan.

Pandas `read_csv(..., chunksize=200)` reads fixed row batches. The file remains on disk. Each valid batch goes through ML/risk before results are sent. Upload itself is a separate operation completed before scanning; results stream during analysis, not during upload.

SSE sends one-way messages over HTTP. Here the frontend uses `fetch` plus `ReadableStream` to parse SSE, so it **can send the normal Authorization header**. The mentor guide's header limitation applies to native EventSource, not every SSE client. We avoid access tokens in URLs because URLs may appear in logs/history.

Messages use IDs and types: start, chunk, heartbeat, error, done. The frontend skips repeated IDs. A bounded queue keeps eight display messages, dropping oldest if a viewer falls behind. Successful

rows are saved before display messages, so export is more complete than a slow viewer's live window. Do not compute final totals from only the screen's 200 rows.

Scoring runs off the event loop. A heartbeat is emitted after about 20 seconds without a message. A recoverable scoring/storage chunk error is logged and reported; the next chunk is attempted. A malformed CSV read that cannot safely resume is fatal; the report says Failed instead of pretending to scan every row.

Stop/disconnection prevents further reads. A current ML call cannot be forcibly cancelled inside a Python thread and may finish. Polling checks for inactivity and stops when the viewer stops requesting results. A new scan always begins at row one. Historical reports are not saved reader bookmarks.

Single worker is deliberate: in-memory scan jobs belong to that process. Multiple processes require routing affinity or shared job messaging such as Redis. Two active scans are permitted; further starts return HTTP 429. This is a small-project concurrency limit, not enterprise scaling.

10. Performance: more than accuracy

Suppose 990 of 1,000 records are benign. A model saying BENIGN every time achieves 99% accuracy and misses every attack. That is a bad security detector.

Metric	Question it answers
Precision	Of flows flagged as attacks, how many were actually attacks?
Recall / detection rate	Of actual attacks, how many were detected?
F1	How well are precision and recall balanced?
Macro F1	How well do classes perform when each gets equal weight?
Weighted F1	How well do classes perform weighted by their example counts?
Per-class recall	Are rare classes such as SQL Injection being missed?
False positive rate	What fraction of benign flows were incorrectly flagged?
Attack average precision	How well attack probability scores rank attacks across thresholds?
Confusion matrix	Which true classes are mistaken for which predicted classes?
Batch latency	How long inference takes for a batch on this machine?
Rows/second	How quickly the processing pipeline handles flows?
Memory RSS	Process memory reported by the operating system

Precision = TP/(TP+FP), recall = TP/(TP+FN), F1 = 2PR/(P+R), false positive rate = FP/(FP+TN). TP is a correctly flagged attack. FP is a benign record flagged as attack. FN is a missed attack. TN is a correctly classified benign record.

Training reports include per-class precision/recall/F1/support and multiclass confusion matrices. They also include binary benign-versus-attack metrics. The UI's model agreement rate is not accuracy, and a risk distribution is not a classification report.

Timing uses five repeated `predict_proba` runs on the holdout batch. A p95 computed from five runs is only a rough sample, not a robust service SLA. Scan timing measures the end-to-end producer including file reading, scoring and result storage. Pipeline chunk latency currently measures preprocessing/inference/row construction, not browser rendering or incident-notification delivery. RSS is operating-system-reported current memory, not peak memory. Isolation Forest review performance is not independently quantified in this sample.

11. What the included results actually show

The supplied CSV has 302 rows covering 16 labels, with roughly 20 examples per class. It deliberately overrepresents attacks relative to most production networks. After duplicate removal, the demo has 234 training rows and 59 holdout rows. Only four holdout rows are benign. A single error changes the benign false positive rate by 25 percentage points.

The included sample-trained RF has macro F1 about 0.890 and false positive rate 0.75. XGBoost has macro F1 about 0.947 and false positive rate 0.0. Extra Trees has macro F1 about 0.866 and false positive rate 1.0. These are measured sample results, not a production ranking. Zero observed false positives among four benign examples is very weak evidence. All models need independent larger validation.

Many rows scanned in the sample demonstration were used in model training. Their scan predictions are not a new test set. The published holdout metrics are the distinct 59-row evaluation. Never say a perfect-looking live scan proves perfect model accuracy.

Your original notebook reported strong full-dataset metrics and already had RF/XGBoost tuning. Those historical results are not regenerated by this package because the full data and original serialized model files were not attached. The tiny demo results must not replace them in your report without explaining the difference.

12. IP addresses and evidence

We validate existing IP values; we do not detect an IP from packet rates. A CSV must preserve Src IP / Source IP and Dst IP / Destination IP columns. These are metadata columns passed through alongside predictions. If absent, unavailable is the honest answer. If invalid, we show invalid rather than making up an address.

The separate demo-IP CSV uses documentation ranges 192.0.2.0/24 and 198.51.100.0/24 with an explicit synthetic provenance column. These only demonstrate display/validation. No geo-IP, WHOIS, reputation feed or attacker attribution is implemented. An IP alone does not prove who performed an attack, particularly behind NAT/proxies or with spoofed traffic.

13. Storage and security controls

Users table stores username, salted password hash, role and creation time. Scans stores owner, source, status and summary. Flows stores successful row payloads, including invalid-row records, with scan/row numbers. Incidents stores label, risk, status and notes. Audit stores who did which tracked action and when.

Prepared SQL parameters prevent values being interpreted as SQL. Paths are confined to the configured CSV directory. Generated upload filenames prevent user-controlled traversal. Analyst-upload ownership is checked when listing/selecting sources. Admin sees all scans; analysts see their own history. Viewer is a read-oriented role with mutation restrictions. The audit log is a teaching application trail, not tamper-proof forensic evidence.

Passwords use 310,000 PBKDF2-SHA256 iterations plus a random salt. The JWT secret is generated and stored privately unless set through JWT_SECRET. Production needs secure secret management, HTTPS and login rate limiting. Tokens expire after two hours. Role changes take effect via a fresh database role check even if an old token remains valid. There is no refresh-token or per-token revocation system.

14. Postman is an API client, not a replacement backend

Swagger UI documents/tests FastAPI routes. Postman is another client that sends requests to the same API, saves collections and makes demo/test sequences easier. The backend remains FastAPI. Read POSTMAN.md and import the included collection. Show login, Bearer authorization, scan creation, streamed events, report, incident update and export.

SSE can be inspected in Postman as an HTTP event stream. Cancel the request to disconnect. Choose polling explicitly if the client/network cannot maintain SSE. Do not start a second scan merely because a stream failed: show the failed scan report, then start a new scan intentionally.

15. A confident five-minute demo

0:00-0:30: Say the introduction and the honest demo-data limitation.

0:30-1:10: Login, select backend CSV, explain flow versus packet. Use the synthetic-IP file only if you state the addresses are synthetic.

1:10-2:10: Click Scan. Explain fixed chunks, SSE and row-one restart. Open one result's evidence and calculate its risk aloud. Explain confidence separately.

2:10-3:00: Open Model laboratory. Compare macro F1, attack recall, false positive rate and latency. Explain the small benign test support.

3:00-3:40: Open Incident queue and save an investigation note. Explain review is human work, not automatic proof or blocking.

3:40-4:20: Open report/export, then repeat scan from row one. Show Postman routes if time permits.

4:20-5:00: State limits and next steps: full data, day/session holdout, original IP preservation, packet capture adapter, UNSW schema adapter and cloud deployment.

16. Likely questions and accurate answers

What does High mean? A rounded priority from 51 to 75 under our configured policy. It is not a universal standard or certainty of attack.

Where does severity come from? A manual class severity table in engine.py; it is not learned and needs domain validation.

Where does confidence come from? The classifier's predict_proba for its chosen class. Uncalibrated confidence may be overconfident.

Why another model? To compare algorithms fairly and expose disagreements, not just to increase code complexity.

Which model is best? We examine macro/per-class F1, recall, false alarms, timing and memory on independent data. The sample ranking is not sufficient to choose a production model.

Why do low-risk records sometimes create incidents? Uncertainty/anomaly is separate from severity-weighted priority. A Low score does not prove benign behaviour.

Is this real-time network capture? Results arrive live while a stored CSV is analyzed. It is near-real-time CSV scanning, not live packet capture.

Why not predict IPs? IPs are source metadata; they cannot be reconstructed from the numeric feature sample.

What happens on restart? Running scans become Interrupted in history. New scan reader opens at row one; no cursor recovery.

Will Stop kill ML instantly? It stops future reading. A current bounded thread call can finish. We do not claim hard real-time interruption.

Why not use only accuracy? Class imbalance can hide missed attacks. Recall, rare-class metrics and false alarms matter.

Why not add a chatbot now? The core analysis and evidence should be validated first. A future LLM should explain supplied evidence, not invent detection results or receive sensitive logs without review.

17. Before October 17

On October 12, prioritize getting the package running and learning risk/streaming. On October 13, bring the full original dataset and train/validate models. On October 14, preserve original IP/protocol metadata and test realistic CSVs. On October 15, rehearse Postman and incident/report workflows; deploy Docker if available. On October 16, rehearse the full demo and prepare screenshots/results with clear provenance. On October 17, demonstrate only verified capabilities and bring a backup local recording/export.

You do not need to pretend to know everything. You need to understand the data path, explain your decisions and state what has and has not been measured.

Reference and implementation sources

Primary project requirements: your uploaded AI_Network Anomaly Detection & Threat Monitoring System PDF and Streaming CSV Data to the App guide. Implementation review: supplied main.py, exploration.ipynb, PROJECT_NOTES.md, CSV and frontend files.

Scikit-learn model evaluation: https://scikit-learn.org/stable/modules/model_evaluation.html
Scikit-learn parameter search: https://scikit-learn.org/stable/modules/grid_search.html
Starlette responses: https://www.starlette.io/responses/
Postman SSE: https://blog.postman.com/support-for-server-sent-events/